

UNIVERSIDAD DE LA COSTA (CUC)

Ingeniería de Sistemas

Asignatura: Desarrollo Web

Grupo: 25911

TALLER (20%)

Panel de Resultados de Fútbol con React

Presentado a: Luis Fernando Toscano Castilla

Presentado por: Diego de Jesús Navarro Gómez

Barranquilla, Colombia

24 de octubre de 2026

Periodo académico: 2026-2

Panel de Resultados de Fútbol con React

Este trabajo presenta una aplicación web hecha con React y Vite para un torneo de fútbol aficionado. La aplicación permite registrar los marcadores de los partidos, consultar el historial y ver la tabla de posiciones, que se calcula automáticamente. Los datos se manejan en memoria durante la ejecución, sin backend.

Entregables

Tabla 1

Ubicación de los entregables

Entregable	Ubicación
Código fuente	Carpeta panel-futbol y repositorio https://github.com/dgonav/panel-futbol
README con instrucciones	Archivo README.md del proyecto
Video de demostración	https://drive.google.com/file/d/1MQOwBkEt5GZFWCSn4zbZvNx_nwRxB_eA/view?usp=sharing
Aplicación desplegada	https://dgonav.github.io/panel-futbol/

Componentes Creados

La aplicación usa componentes funcionales y Hooks (Meta Open Source, s.f.). Los seis equipos están en `src/datos.js` con un `id` único, `nombre` y `escudo`; los escudos son de diseño propio en formato SVG. La Tabla 2 resume los componentes.

Tabla 2

Componentes de la aplicación

Componente	Función
App	Componente principal. Guarda la lista de partidos y las funciones para agregar y eliminar.
ListaEquipos	Muestra los equipos en tarjetas con su escudo.
FormularioPartido	Formulario para elegir local y visitante e ingresar el marcador. Valida los datos y muestra el último resultado.

Componente	Función
Resultado	Muestra un partido con escudos, nombres y marcador. Se usa en el formulario y en el historial.
Historial	Lista los partidos, permite filtrarlos por equipo y eliminarlos.
TablaPosiciones	Calcula y ordena la tabla de posiciones.

Manejo del Estado

El estado se maneja con `useState` en tres componentes:

- En `App`: `partidos`, un array donde cada partido tiene `id`, `localId`, `visitanteId`, `golesLocal` y `golesVisitante`. Para agregar se copia el array y se le suma el partido nuevo; para eliminar se usa `filter()`:

```
setPartidos([...partidos, { ...partido, id: Date.now() }])
setPartidos(partidos.filter((p) => p.id !== id))
```

- En `FormularioPartido`: los valores de los campos, el mensaje de `error` y el `ultimo` partido registrado.
- En `Historial`: el `filtro` por equipo.

La tabla no tiene estado propio: se calcula cada vez a partir de `partidos`, que le llega por props. Por eso se actualiza sola cuando se registra o se elimina un partido. Cuando no hay partidos se muestra el mensaje "No hay partidos registrados".

Validaciones

Al enviar el formulario (evento `onSubmit`) se revisa con JavaScript que se hayan elegido los dos equipos, que no sea el mismo equipo en ambos lados y que los marcadores sean números enteros no negativos. Para esto último se usa la expresión regular `/^\d+$/`, que no acepta negativos, decimales ni campos vacíos (MDN Web Docs, s.f.-b). Si algo falla, se muestra un mensaje en rojo y el partido no se guarda.

Cálculos Realizados

Para cada equipo se recorren los partidos con `reduce()` (MDN Web Docs, s.f.-a). Si el equipo jugó el partido se suman los partidos jugados (PJ), ganados (PG), empatados (PE) y perdidos (PP), y los goles a favor (GF) y en contra (GC). Después se calculan:

- Diferencia de goles: `dg = gf - gc`.
- Puntos: `pts = pg * 3 + pe * 1`; la derrota suma cero.

La tabla se ordena con `sort()` por puntos, luego por diferencia de goles y luego por goles a favor:

```
.sort((a, b) => b.pts - a.pts || b.dg - a.dg || b.gf - a.gf)
```

Listas, Eventos y Diseño Adaptable

Las listas (equipos, opciones, historial y filas de la tabla) se muestran con `map()` usando el `id` como `key`. Se usan los eventos `onChange` en los campos, `onSubmit` en el formulario y `onClick` en el botón Eliminar. Con CSS Grid y una regla `@media (max-width: 700px)` la página pasa a una sola columna en el celular.

Referencias

MDN Web Docs. (s.f.-a). *Array.prototype.reduce()*. Mozilla. https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce

MDN Web Docs. (s.f.-b). *Expresiones regulares*. Mozilla. https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular_expressions

Meta Open Source. (s.f.). *React: Documentación oficial*. https://react.dev/learn